PYTHON FOR ACTUARIES · NOTEBOOK 06 · SOLUTION

Exploring Motor Insurance Claims

Hands-on Exercise

Explore `vehicle_fraud.csv` on your own with the tools from Notebooks 03, 04 and 05. Work through three phases:

- **Phase 1**: The Portfolio
- **Phase 2**: Claim Amounts
- **Phase 3**: Fraud

The hints name the tools; the code is up to you.

Several tasks repeat steps from the exercises in Notebooks 03 to 05 on purpose: here they come together in one exploration. Your code from there can serve as a starting point.

The Dataset: `vehicle_fraud.csv`

10,050 rows of synthetic motor insurance claims:

Column	Description
`claim_id`	Claim number
`policyholder_age`	Age in years
`occupation`	`Employed`, `Self-employed`, `Unemployed`, `Student`
`region`	Region `R01` to `R10`
`credit_score`	Credit score (300–850)
`customer_since_years`	Years as a customer
`vehicle_value`	Vehicle value
`vehicle_type`	`Compact`, `Mid-range`, `Luxury`, `SUV`, `Sports car`
`coverage_type`	`Liability`, `Partial coverage`, `Comprehensive`
`number_of_prior_claims`	Number of earlier claims
`claim_amount`	Claim amount
`police_report`	`1` if a police report was filed
`witnesses_present`	`1` if witnesses were present
`reporting_delay_days`	Days between accident and claim report
`fraud`	`1` = fraudulent, `0` = legitimate

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Seaborn style for all plots
sns.set_theme(style='whitegrid')

# Avoid truncation of DataFrame output in Jupyter
pd.set_option('display.expand_frame_repr', False)

# Display all columns in the DataFrame output
pd.set_option('display.max_columns', None)

# Define a separator for better readability
sep = "=" * 50
```

The next two cells load the raw export and clean it as in Notebook 05.

```
# Load the dataset
df = pd.read_csv('vehicle_fraud.csv')
```

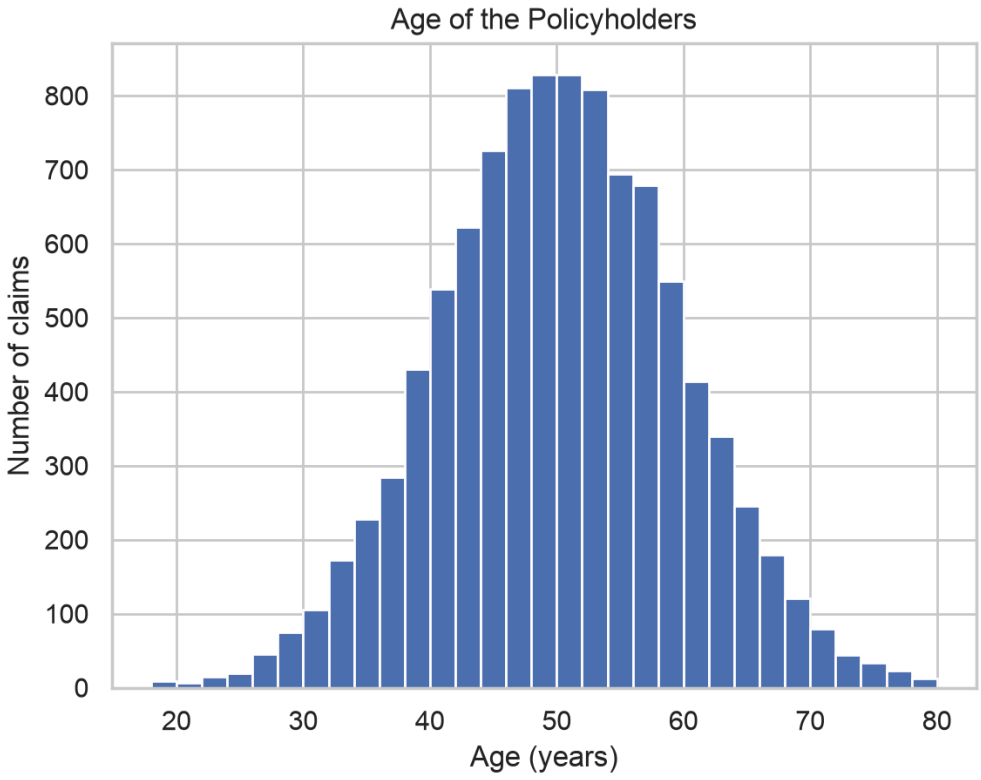
```
# Clean as in Notebook 05: remove the duplicates, the placeholder age 0 becomes NaN
df = df.drop_duplicates()
df.loc[df['policyholder_age'] < 18, 'policyholder_age'] = np.nan
```

Phase 1: The Portfolio

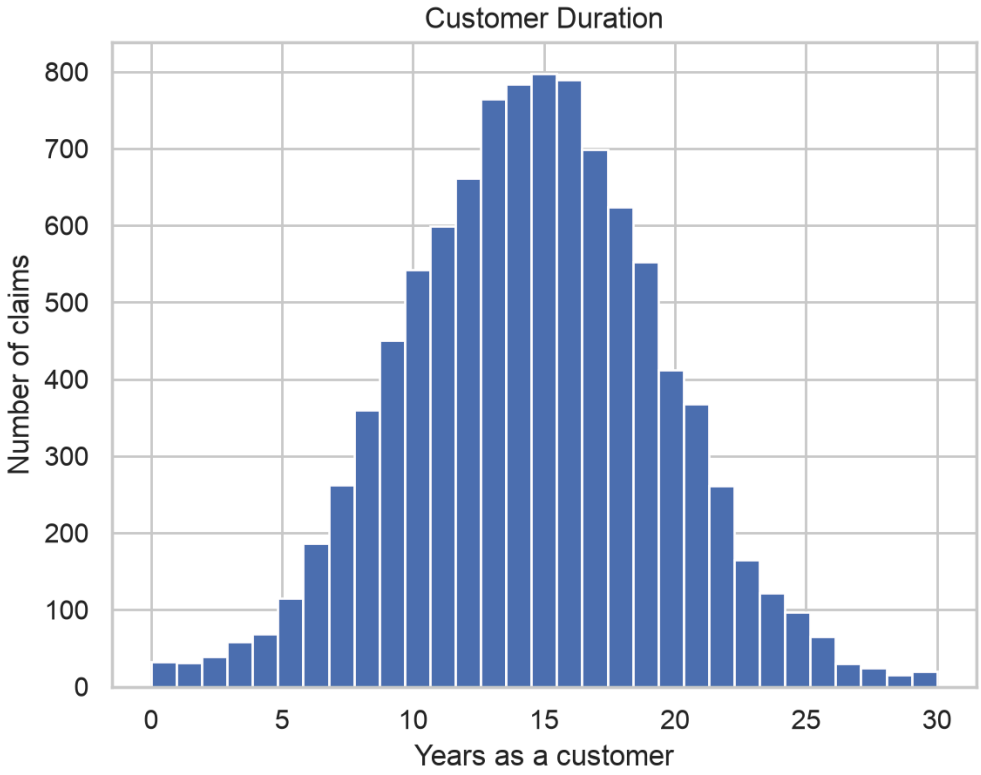
Get to know the insured customers and vehicles: ages, customer durations, occupations, regions, vehicle types and coverage types.

Task 1: Plot histograms of `policyholder_age` and `customer_since_years`. Which ages and customer durations are typical?

```
# Task 1: Histogram of the age
df['policyholder_age'].plot(kind='hist', bins=31)
plt.title('Age of the Policyholders')
plt.xlabel('Age (years)')
plt.ylabel('Number of claims')
plt.show()
```



```python
# Task 1: Histogram of the customer duration
df['customer_since_years'].plot(kind='hist', bins=31)
plt.title('Customer Duration')
plt.xlabel('Years as a customer')
plt.ylabel('Number of claims')
plt.show()
```



Task 2: What share of the claims comes from each `occupation` and each `region`?

```python
# Task 2: Shares of the occupations
print(f"\n{sep}\nShares of Occupations:
    df['occupation'].value_counts(normalize=True)\n{sep}")
print(df['occupation'].value_counts(normalize=True).round(3))
```

```
================================================
Shares of Occupations: df['occupation'].value_counts(normalize=True)
================================================
occupation
Unemployed       0.255
Self-employed    0.252
Employed         0.248
Student          0.245
Name: proportion, dtype: float64
```

```python
# Task 2: Shares of the regions
print(f"\n{sep}\nShares of Regions: df['region'].value_counts(normalize=True)\n{sep}")
print(df['region'].value_counts(normalize=True).sort_index().round(3))
```

```
==================================================
Shares of Regions: df['region'].value_counts(normalize=True)
==================================================
region
R01    0.103
R02    0.097
R03    0.101
R04    0.097
R05    0.100
R06    0.099
R07    0.103
R08    0.094
R09    0.100
R10    0.106
Name: proportion, dtype: float64
```

Task 3: Count the claims per `vehicle_type` and `coverage_type` in one table. Which vehicle types are mostly fully insured?

```python
# Task 3: Claims per vehicle type and coverage type
print(f"\n{sep}\nGroup Sizes: df.groupby(['vehicle_type', 'coverage_type'])
    ['claim_id'].count().unstack()\n{sep}")
print(df.groupby(['vehicle_type', 'coverage_type'])['claim_id'].count().unstack())
```

```
==================================================
Group Sizes: df.groupby(['vehicle_type', 'coverage_type'])['claim_id'].count().unstack()
==================================================
coverage_type  Comprehensive  Liability  Partial coverage
vehicle_type
Compact                  683        628               671
Luxury                  1445        294               273
Mid-range                826        608               605
SUV                     1063        445               494
Sports car              1273        369               323
```

Most policyholders are between 40 and 60 years old, and most have been customers for 10 to 20 years. Both columns hold whole years: with 31 bins, each bin of the age covers two years and each bin of the customer duration one year. With 30 bins, some bins would cover one year more than their neighbours and show a false peak.

Occupations and regions are evenly spread: each occupation has about 25% of the claims, each region about 10%. Luxury cars (72%), sports cars (65%) and SUVs (53%) are mostly fully insured, compact cars only in 34% of the claims.

Phase 2: Claim Amounts

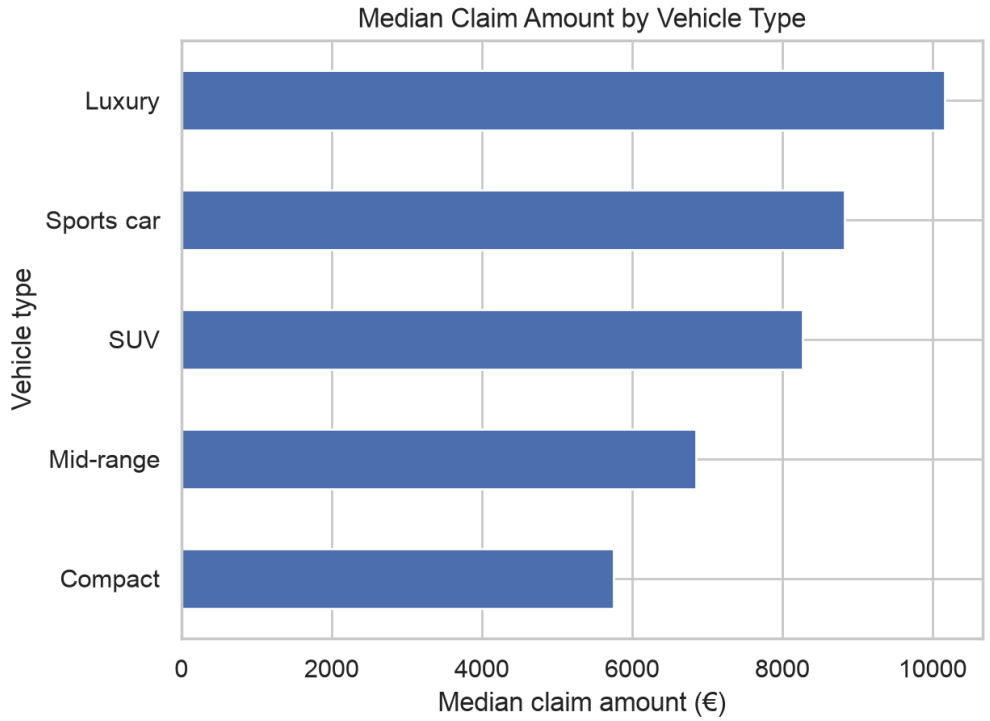
Compare the claim amounts between groups and find the features with the largest differences.

Claim amounts are right-skewed (Notebook 04), so the median describes a typical claim better than the mean.

Task 1: Compute the median claim amount per `vehicle_type`, `coverage_type` and `region` and plot each as a horizontal bar chart. Which feature shows the largest differences?

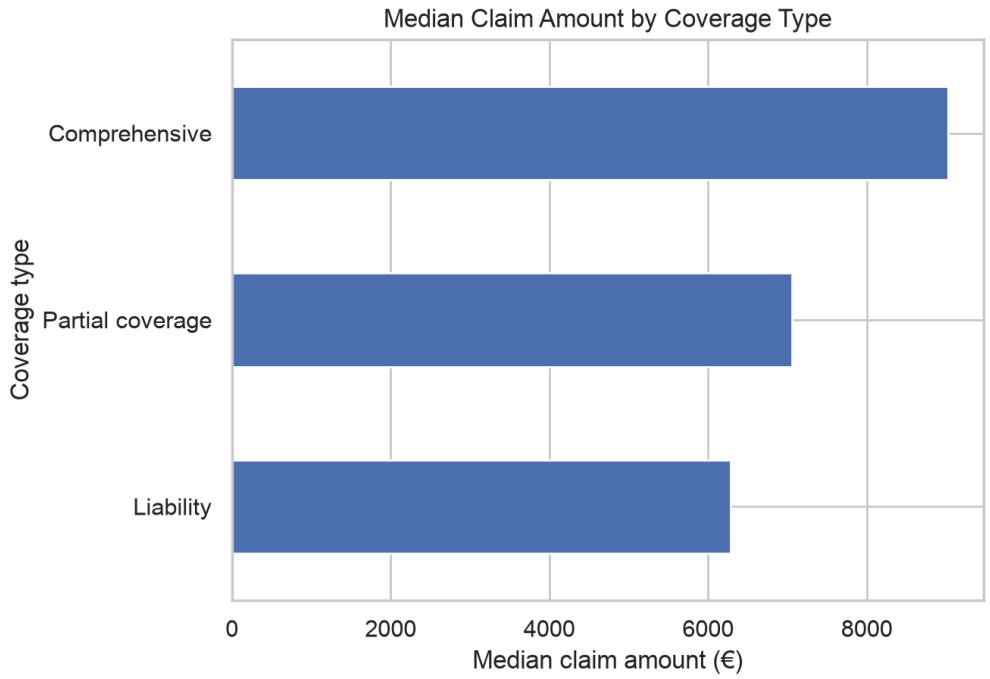
```
# Task 1: Median claim amount per vehicle type
median_by_type = df.groupby('vehicle_type')['claim_amount'].median().sort_values()

median_by_type.plot(kind='barh')
plt.title('Median Claim Amount by Vehicle Type')
plt.xlabel('Median claim amount (€)')
plt.ylabel('Vehicle type')
plt.show()
```



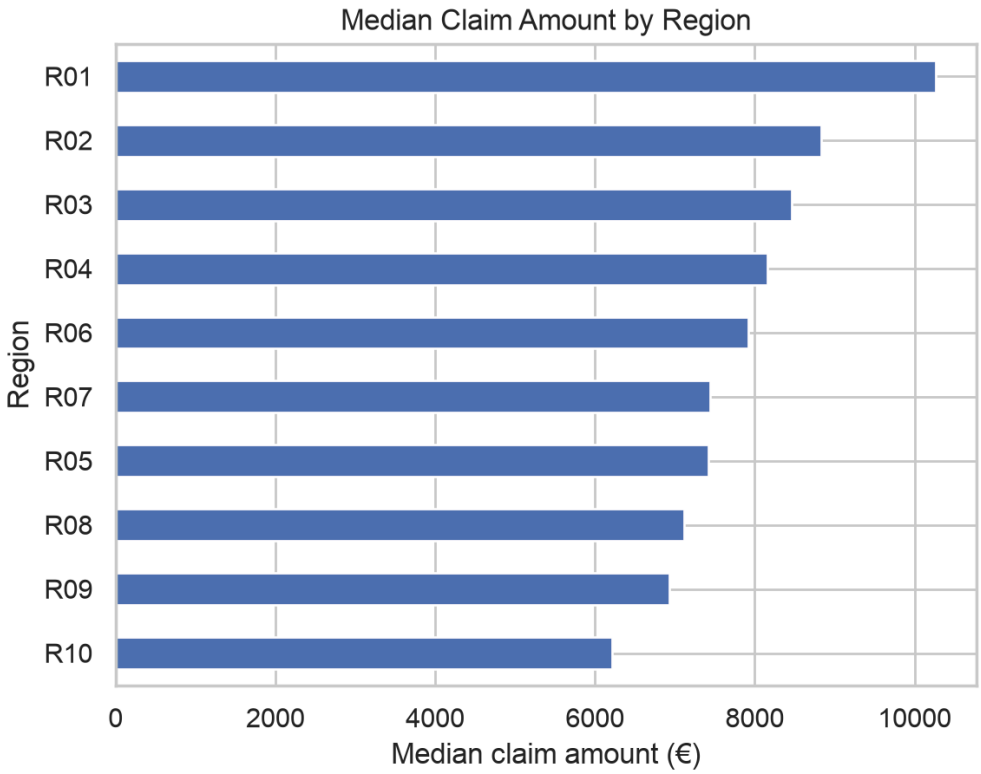
```python
# Task 1: Median claim amount per coverage type
median_by_coverage = df.groupby('coverage_type')['claim_amount'].median().sort_values()

median_by_coverage.plot(kind='barh')
plt.title('Median Claim Amount by Coverage Type')
plt.xlabel('Median claim amount (€)')
plt.ylabel('Coverage type')
plt.show()
```



```python
# Task 1: Median claim amount per region
median_by_region = df.groupby('region')['claim_amount'].median().sort_values()

median_by_region.plot(kind='barh')
plt.title('Median Claim Amount by Region')
plt.xlabel('Median claim amount (€)')
plt.ylabel('Region')
plt.show()
```

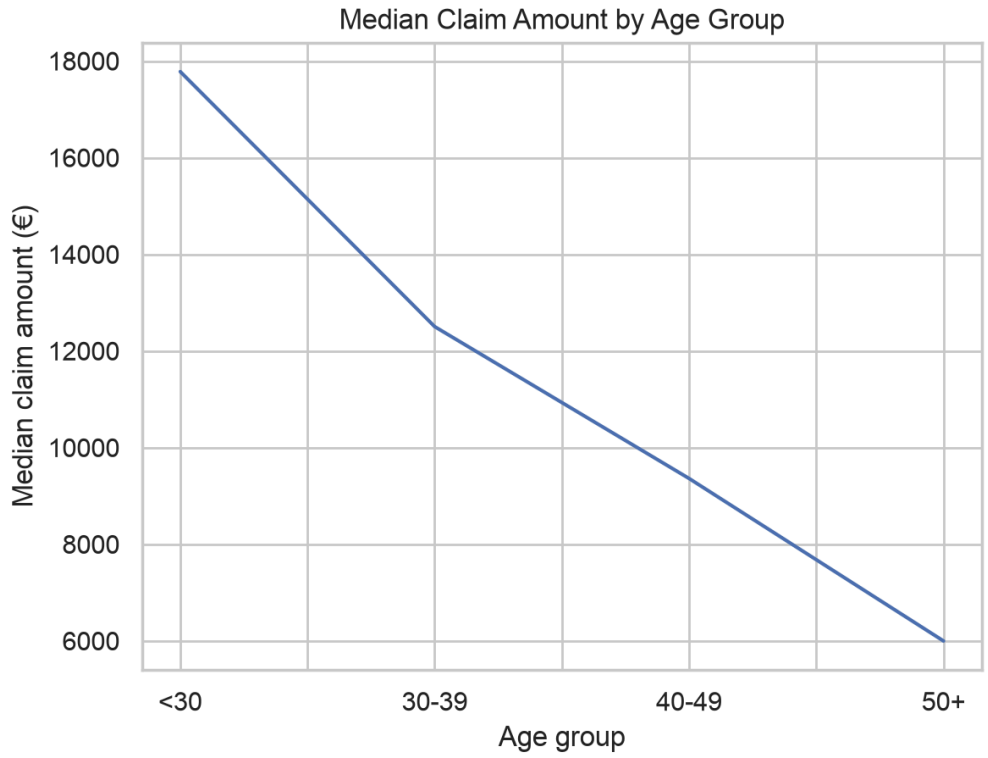


Task 2: Create the column `age_group` as in Notebook 03 (`<30`, `30-39`, `40-49`, `50+`). Plot the median claim amount per age group. How does it change with age?

```python
# Task 2: Age groups as in Notebook 03
df['age_group'] = pd.cut(df['policyholder_age'], bins=[0, 30, 40, 50, 100], right=False,
                         labels=['<30', '30-39', '40-49', '50+'])
```

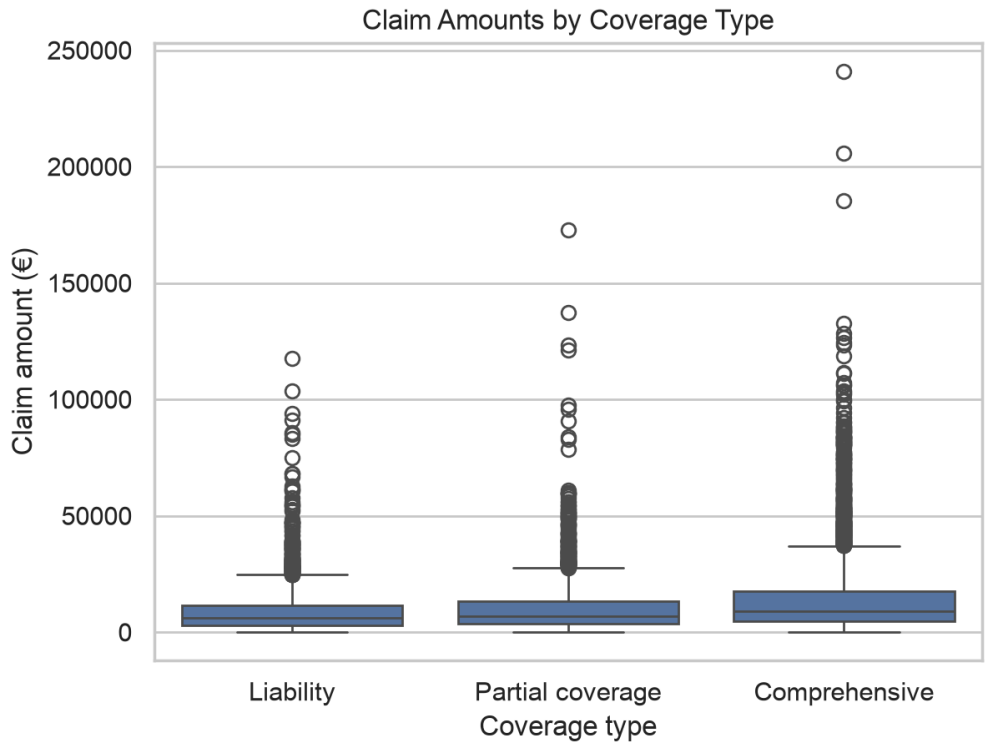
```python
# Task 2: Median claim amount per age group
median_by_age = df.groupby('age_group', observed=True)['claim_amount'].median()

median_by_age.plot(kind='line')
plt.title('Median Claim Amount by Age Group')
plt.xlabel('Age group')
plt.ylabel('Median claim amount (€)')
plt.show()
```



Task 3: Draw a box plot of `claim_amount` per `coverage_type`. What do the points above the whiskers stand for?

```
# Task 3: Box plot of the claim amounts per coverage type
sns.boxplot(data=df, x='coverage_type', y='claim_amount')
plt.title('Claim Amounts by Coverage Type')
plt.xlabel('Coverage type')
plt.ylabel('Claim amount (€)')
plt.show()
```



All three features show clear differences. The median claim of a luxury car is 10,156 € compared with 5,755 € for a compact car, the largest difference of the three. Comprehensive cover has a median of 9,018 € compared with 6,279 € for liability-only cover, `R01` 10,260 € compared with 6,218 € in `R10`. The age shows an even larger difference: the median falls from 17,782 € under 30 to 5,999 € at 50 and over. The points above the whiskers are large claims, genuine values rather than errors (Notebook 05).

Luxury cars are mostly fully insured (Phase 1), so part of their higher claim amounts may come from the coverage. Notebook 08 separates such effects with a GLM.

Phase 3: Fraud

Find out in which groups fraud is frequent and in which it is not.

`fraud` is `1` for a fraudulent claim, so its mean per group is the fraud rate of the group (Notebook 03).

Task 1: Compute the fraud rate per `vehicle_type`, `coverage_type`, `region` and `occupation`. Does any of these features separate fraudulent from legitimate claims?

```
# Task 1: Fraud rate per group of four features
for column in ['vehicle_type', 'coverage_type', 'region', 'occupation']:
    print(f"\n{sep}\nFraud Rate by {column}: df.groupby('{column}')['fraud'].mean()\n{sep}")
    print(df.groupby(column)['fraud'].mean().round(3))
```

```
==================================================
Fraud Rate by vehicle_type: df.groupby('vehicle_type')['fraud'].mean()
==================================================
vehicle_type
Compact       0.117
Luxury        0.123
Mid-range     0.115
SUV           0.120
Sports car    0.118
Name: fraud, dtype: float64

==================================================
Fraud Rate by coverage_type: df.groupby('coverage_type')['fraud'].mean()
==================================================
coverage_type
Comprehensive       0.119
Liability           0.118
Partial coverage    0.118
Name: fraud, dtype: float64

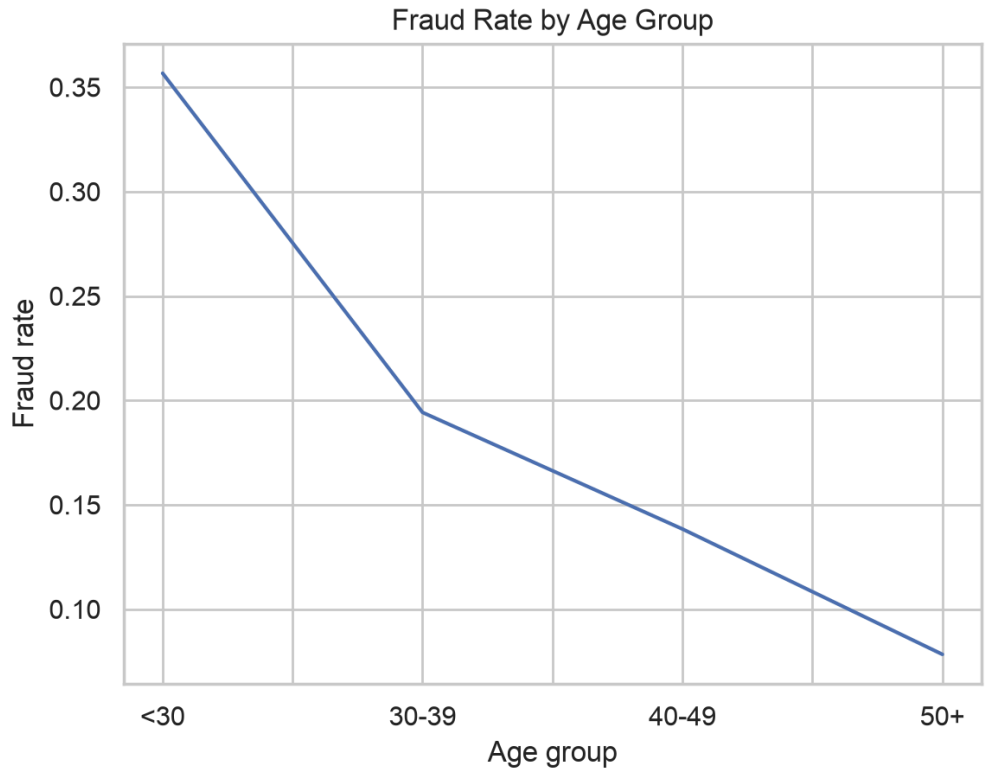
==================================================
Fraud Rate by region: df.groupby('region')['fraud'].mean()
==================================================
region
R01    0.125
R02    0.129
R03    0.097
R04    0.135
R05    0.111
R06    0.106
R07    0.131
R08    0.118
R09    0.120
R10    0.112
Name: fraud, dtype: float64

==================================================
Fraud Rate by occupation: df.groupby('occupation')['fraud'].mean()
==================================================
occupation
Employed         0.119
Self-employed    0.101
Student          0.126
Unemployed       0.127
Name: fraud, dtype: float64
```

Task 2: Plot the fraud rate per age group and per customer duration band: under 5, 5–9, 10–19 and 20 or more years as a customer.

```
# Task 2: Fraud rate per age group
fraud_by_age = df.groupby('age_group', observed=True)['fraud'].mean()

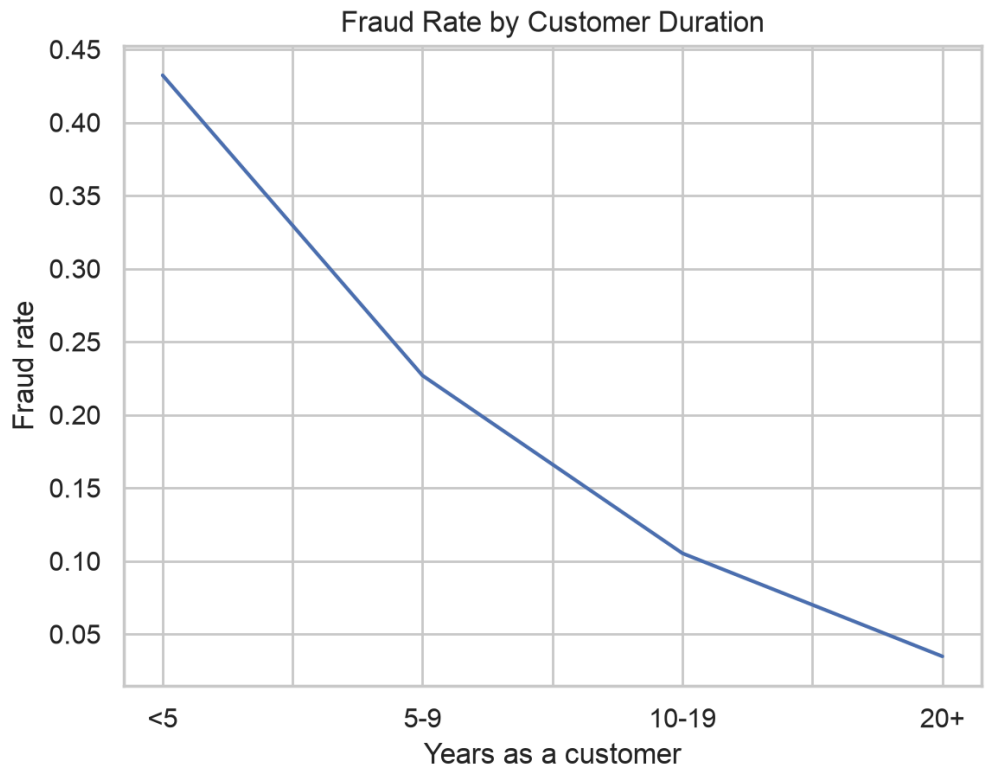
fraud_by_age.plot(kind='line')
plt.title('Fraud Rate by Age Group')
plt.xlabel('Age group')
plt.ylabel('Fraud rate')
plt.show()
```




```
# Task 2: Customer duration bands
df['customer_band'] = pd.cut(df['customer_since_years'], bins=[0, 5, 10, 20, 31],
    right=False,
                             labels=['<5', '5-9', '10-19', '20+'])
```

```
# Task 2: Fraud rate per customer duration band
fraud_by_customer_band = df.groupby('customer_band', observed=True)['fraud'].mean()

fraud_by_customer_band.plot(kind='line')
plt.title('Fraud Rate by Customer Duration')
plt.xlabel('Years as a customer')
plt.ylabel('Fraud rate')
plt.show()
```



Task 3: Compute the fraud rate per vehicle value band: under 10,000 €, 10,000–20,000 €, 20,000–40,000 € and 40,000 € or more. For which vehicles is fraud most frequent?

```
# Task 3: Vehicle value bands
df['value_band'] = pd.cut(df['vehicle_value'], bins=[0, 10000, 20000, 40000, 80000],
    right=False,
                          labels=['<10k', '10k-20k', '20k-40k', '40k+'])
```

```
# Task 3: Fraud rate per vehicle value band
print(f"\n{sep}\nFraud Rate by Vehicle Value: df.groupby('value_band', observed=True)
    ['fraud'].mean()\n{sep}")
print(df.groupby('value_band', observed=True)['fraud'].mean().round(3))
```

```
==================================================
Fraud Rate by Vehicle Value: df.groupby('value_band', observed=True)['fraud'].mean()
==================================================
value_band
<10k       0.544
10k-20k    0.194
20k-40k    0.079
40k+       0.022
Name: fraud, dtype: float64
```

The fraud rate is between 10% and 14% for every vehicle type, coverage type, region and occupation: these features do not separate fraud. Fraud is much more frequent for young policyholders (36% under 30, 8% at 50 and over), for new customers (43% under 5 years, 4% at 20 years or more) and for cheap vehicles (54% under 10,000 €, 2% at 40,000 € or more).

Conclusion

Task: Summarize your findings in a few sentences: Which features drive the claim amount? Which features point to fraud, and which do not?

Vehicle type, coverage type, region and age drive the claim amount. Fraud does not depend on the product features but on the policyholder and the vehicle: young policyholders, new customers and cheap vehicles show high fraud rates. Notebook 07 trains models that learn such patterns, and Notebook 09 uses them to build a complete fraud detection model.

Summary

- **Portfolio:** histograms, shares with `value_counts(normalize=True)`, counts per combination with `groupby()` and `unstack()`
- **Claim amounts:** median per group, bar charts, line plots and box plots
- **Fraud:** fraud rate per group, groups from `pd.cut()`

Next: training machine learning models in Notebook 07.